University Management System – README'

Project Overview

This project extends a University Course Registration System using advanced Object-Oriented Programming concepts, including generic programming, inheritance, and exception handling. In this system, one can have various user roles like Administrators, Professors, Students, and Teaching Assistants (TAs), which may be able to interact with courses and perform actions like course registration, grade administration, and submission of feedback.

This version introduces new features such as Generic Feedback System, Enhanced User Role Management, and Robust Exception Handling integrated in the previous existing system functionalities.

Assumptions

Role-Based Access: All the user types like Admin, Professor, Student, TA have roles which have their respective functionalities. Functionalities are also limited to the role; for example, a Student cannot access professor's actions.

Course Feedback: A student is able to comment only on a course he has already completed, and the comments are stored within a generic system that accepts numeric feedback as well as textual comments.

TA Role: Teaching Assistants can support professors in respect of grade handling but have fewer privileges than Professors, like changing course information.

Limit and Enrollment Dates: Courses have capacity limits and drop dates both of which are thrown by exception handling

Exceptions: Login failed, course full, and a drop date for a course, all these is handled using user defined exceptions to avoid system crashes

Object Oriented Concepts Used

1. Classes and Objects

The system is implemented through a number of classes implementing various kinds of users and entities at the university:

Users. The system defines classes Admin, Professor, Student, and Teaching Assistant, inheriting from some base class User.

Courses. The Course class captures details on individual courses: course id, name, students enrolled, and professors assigned.

Course Registration and Comments: Techniques in the Class class handle student registration, grade assignment and the storage of comments.

An object of these classes are created during runtime for each user and course to allow various operations within the system.

2. Inheritance

Inheritance is the way in which the TA class is declared. TAs, being a subclass of the Student class, inherit all functionalities of the student: they can enroll in courses and view grades. Furthermore, TAs can administrate student grades on courses assigned to them but cannot update course details like a professor could. This is a demonstration of inheritance - a TAs is a specific student with extra functionalities.

3. Polymorphism

A program demonstrates polymorphism when you think about how a user may belong to any of several classes: Admin, Professor, Student, or TA, but you treat users uniformly by using a common base class User. As such, login functionality treats all those types of users under one interface but the specific actions depend on their actual type-for instance, a professor can assign grades while a student cannot.

4. Encapsulation

Encapsulation is realized by indirect, rather than direct, access to sensitive information like users' credentials and course-related information. Each class lets out only the necessary methods of its API (such as getters and setters) while hiding any internal state logic.

This ensures data integrity and prevents modification under unauthorized or unintentional circumstances.

5. Generic Programming

A Generic Feedback System allows students to give feedback for courses that have been completed. This can be numeric ratings, such as 1-5, or textual comments. A generic class has been developed to take care of feedback and ensures that it is type-safe at different data types. This gives flexibility without losing robustness.

6. Exception Handling

The system employs custom exception classes to handle a variety of error conditions. The use of exceptions ensures that, in case of errors, the program handles them politely instead of crashing unapologetically. Key scenarios where exception handling is applied:

Invalid Login: When a user enters incorrect login credentials, the system throws a custom InvalidLoginException, and the system requests the user to reenter their credentials.

Course Full: This exception occurs when it is tried to enroll a student into a course after it is filled to the capacity, and more students cannot be enrolled. None of the remaining enrollments can be completed. Drop Deadline Passed: If it is tried to drop a course after the drop deadline, DropDeadlinePassedException is raised.

These exceptions are caught by try-catch blocks in the program and handled appropriately, giving meaningful error messages to users and ensuring that the system does not terminate abruptly.

New Features and Enhancements

Generic Feedback System:

Students can rate any course they have attended with this system, along with the possibility of giving textual comments. This is a generic feedback class, therefore able to hold different kinds of data in a type-safe manner. Professors can view this feedback through their own user interface.

Teaching Assistant Role:

We have created a new type of user: Teaching Assistant (TA).

TAs have the student functionality (for instance, to enroll themselves in courses) but also some additional privileges to assist professors with administering grades.

TAs cannot perform some of the actions that are forbidden for professors, like updating course information; they are, therefore distinct from students and professors.

Exception Handling:

Custom exceptions help dealing with errors during events like failed login attempts, courses becoming fully enrolled, and dropping late courses.

These exceptions save the system from crashes and also give a clear feedback to a user about failure of any operation.

Conclusion

This advanced version of the University Management System rightly implements all the principles of Object Oriented Programming. Inheritance, Polymorphism, Encapsulation are some of these principles which are well represented along with Generic Programming. The exception handling technique ensures a stable system and always helpful to the user irrespective of errors or invalid operations.

It has implemented the Generic Feedback System and the Teaching Assistant Role in the system as this project adds new features while demonstrating the utility of OOP for changing requirements in maintaining clean, reusable code.